

lsmeear: selección de variables con multiplicadores duales

J Glob Optim (2017)

Ignacio Araya Bertrand Neveu

El problema es optimización global continua

- Se minimiza $f(x)$ sobre una caja $x \in \mathbb{R}^n$.
- Las restricciones son $g(x) \leq 0$ y pueden ser no convexas.
- El solver construye un árbol de búsqueda con **bisección** y **poda**.
- Cada bisección divide una caja en dos subcajas.
- La **variable elegida** influye en el tamaño del árbol.

Smear ignora la importancia de cada restricción

- Las estrategias clásicas agregan impactos por restricción.
- Tratan todas las restricciones como igualmente importantes.
- Eso diluye restricciones activas y sobrevalora las inactivas.
- La selección resultante puede ser poco informativa.

Los multiplicadores duales miden relevancia

- La **función de Lagrange** combina objetivo y restricciones con pesos duales.

-

$$L(x, \lambda) = f(x) + \sum_{j=1}^m \lambda_j g_j(x)$$

- Esta expresión suma $f(x)$ y las restricciones $g_j(x) \leq 0$.
- λ_j^* es el multiplicador dual óptimo asociado a la restricción j .
- Las restricciones inactivas tienen $\lambda_j^* = 0$.
- Los duales ponderan el cambio del óptimo ante variaciones en cada restricción.

lsmeas linealiza y luego aplica smear

- Lineariza cada g_j en el punto medio de la caja actual.
- Resuelve el programa lineal con las restricciones de la caja.
- Usa λ^* como solución dual óptima de esa linealización.
- Calcula $|D_i \text{wid}(x_i)|$ para cada variable.
- Elige la variable con el máximo impacto para bisección.

Si el LP falla, `lsmeas` cae a `smearsum`

- Si la linealización no tiene solución o es ilimitada, `lsmeas` usa `smearsum`.
- La fase 1 es barata: resuelve un LP lineal, no el problema original.
- La selección variable sigue la arquitectura original: linealizar, evaluar, bisecar.
- No depende del **contratista** de relajación convexa usado por el solver.
- Con ello, `lsmeas` mantiene una estrategia robusta y simple.

lsmeiar-MG iguala o mejora a lsmeiar

Variante	Tiempo total	Ganancia total	Lectura
lsmeiar	21 035	0.91	Base
lsmeiar-MG	19 181	1.07	Menor complejidad
lsmeiar-LI	20 429	0.94	Peor total
lsmeiar-LRI	20 429	0.94	Peor total
ssum	–	–	Superado
ssum-active	–	–	Superado

- En la Tabla 2, lsmeiar-MG logra el mejor tiempo total.
- lsmeiar-LI y lsmeiar-LRI quedan claramente por detrás.
- lsmeiar-MG conserva la idea central con menos complejidad.
- Su ganancia total supera a ssum y ssum-active.

lsmeas supera a las estrategias clásicas

Estrategia	Tiempo total	Nodos
lsmeas	mejor	baja en varias instancias
lf	peor entre las clásicas	muchas veces mayor
ssr / smax / ssum / rr	comparables, pero menos favorables	—

- En la Tabla 1, lsmeas reduce tiempos en muchos casos difíciles.
- lf suele ser la estrategia clásica con peor desempeño.
- lsmeas alcanza el mejor tiempo total entre las comparadas.
- El número de nodos también baja en varias instancias.

lsmeas-MG mantiene la ventaja frente a referencias

- La Tabla 3 confirma mejores tiempos totales con lsmeas-MG.
- Frente a minLB y FeasibleDiving, lsmeas-MG conserva ventaja.
- La comparación valida que el efecto no depende solo del buscador.
- En ambos buscadores, lsmeas-MG sigue siendo competitiva.
- Los mejores tiempos de la tabla aparecen, en su mayoría, con lsmeas-MG.

La conclusión es simple: ponderar ayuda

- `lsmear` usa **información dual** para priorizar variables.
- Pondera las restricciones mediante los multiplicadores lagrangianos óptimos.
- Mejora el **branch and bound** intervalar sin cambios profundos.
- `lsmear-MG` logra el mejor equilibrio entre rendimiento y simplicidad.
- Frente a `ViolationTransfer`, es menos intrusivo y no cambia la estrategia general.
- Futuro: añadir la violación de restricciones como señal adicional.